

Test Plan Template is a detailed document that describes the test strategy, objectives, schedule, estimation and deliverables, and resources required for testing. Test Plan helps us determine the effort needed to validate the quality of the application under test. The test plan serves as a blueprint to conduct software testing activities as a defined process which is minutely monitored and controlled by the test manager.

Creating a [Test Plan](#) is mandatory to ensure success of your Software testing project. If you are new to Test Planning refer this tutorial on [How to Create a Test Plan](#)

Test Plan Template

Below find important constituents of a test plan-

- [1 Introduction](#)
 - [1.1 Scope](#)
 - [1.1.1 In Scope](#)
 - [1.1.2 Out of Scope](#)
 - [1.2 Quality Objective](#)
 - [1.3 Roles and Responsibilities](#)
- [2 Test Methodology](#)
 - [2.1 Overview](#)
 - [2.2 Test Levels](#)
 - [2.3 Bug Triage](#)
 - [2.4 Suspension Criteria and Resumption Requirements](#)
 - [2.5 Test Completeness](#)
- [3 Test Deliverables](#)
- [4 Resource & Environment Needs](#)
 - [4.1 Testing Tools](#)
 - [4.2 Test Environment](#)

1) Introduction

Brief introduction of the test strategies, process, workflow and methodologies used for the project

1.1) Scope

1.1.1) In Scope

Scope defines the features, functional or non-functional requirements of the software that **will be** tested

1.1.2) Out of Scope

Out Of Scope defines the features, functional or non-functional requirements of the software that **will NOT be** tested

1.2) Quality Objective

Here make a mention of the overall objective that you plan to achieve with your manual testing and automation testing.

Some objectives of your testing project could be

- Ensure the Application Under Test conforms to functional and non-functional requirements
- Ensure the AUT meets the quality specifications defined by the client
- Bugs/issues are identified and fixed before go live

1.3) Roles and Responsibilities

Detail description of the Roles and responsibilities of different team members like

- QA Analyst
- Test Manager
- Configuration Manager
- Developers
- Installation Team

Amongst others

2) Test Methodology

2.1) Overview

Mention the reason of adopting a particular test methodology for the project. The test methodology selected for the project could be

- WaterFall
- Iterative
- Agile
- Extreme Programming

The methodology selected depends on multiple factors. You can read about Test Methodology [here](#)

2.2) Test Levels

Test Levels define the Types of Testing to be executed on the Application Under Test (AUT). The Testing Levels primarily depends on the scope of the project, time and budget constraints.

2.3) Bug Triage

The goal of the triage is to

- To define the type of resolution for each bug
- To prioritize bugs and determine a schedule for all “To Be Fixed Bugs”.

2.4) Suspension Criteria and Resumption Requirements

Suspension criteria define the criteria to be used to suspend all or part of the testing procedure while Resumption criteria determine when testing can resume after it has been suspended

2.5) Test Completeness

Here you define the criterias that will deem your testing complete.

For instance, a few criteria to check Test Completeness would be

- 100% test coverage
- All Manual & Automated Test cases executed
- All open bugs are fixed or will be fixed in next release

3) Test Deliverables

Here mention all the Test Artifacts that will be delivered during different phases of the testing lifecycle.

Here are the simple deliverables

- Test Plan
- Test Cases
- Requirement Traceability Matrix
- Bug Reports
- Test Strategy
- Test Metrics
- Customer Sign Off

4) Resource & Environment Needs

4.1) Testing Tools

Make a list of Tools like

- Requirements Tracking Tool
- Bug Tracking Tool
- Automation Tools

Required to test the project

4.2) Test Environment

It mentions the minimum **hardware** requirements that will be used to test the Application.

Following **software's** are required in addition to client-specific software.

- Windows 8 and above
- Office 2013 and above
- MS Exchange, etc.

5) Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Program Interface
AUT	Application Under Test

[Download the above Test Plan Template Format](#)

Sample Test Plan Document Banking Web Application Example

1 Introduction

The Test Plan is designed to prescribe the scope, approach, resources, and schedule of all testing activities of the project Guru99 Bank.

The plan identify the items to be tested, the features to be tested, the types of testing to be performed, the personnel responsible for testing, the resources and schedule required to complete testing, and the risks associated with the plan.

1.1 Scope

1.1.1 In Scope

All the feature of websiteGuru99 Bank which were defined in software requirement [specs](#) are need to be tested

Module Name	Applicable Roles	Description
Balance Enquiry	Manager Customer	Customer: A customer can have multiple bank accounts. He can view balance of his accounts only Manager: A manager can view balance of all the customers who come under his supervision
Fund Transfer	Manager Customer	Customer: A customer can have transfer funds from his “own” account to any destination account. Manager: A manager can transfer funds from any source bank account to destination account
Mini Statement	Manager Customer	A Mini statement will show last 5 transactions of an account Customer: A customer can see mini-statement of only his “own” accounts Manager: A manager can see mini-statement of any account
Customized Statement	Manager Customer	A customized statement allows you to filter and display transactions in an account based on date, transaction value Customer: A customer can see Customized-statement of only his “own” accounts Manager: A manager can see Customized -statement of any account

Change Password	Manager Customer	Customer: A customer can change password of only his account. Manager: A manager can change password of only his account. He cannot change passwords of his customers
New Customer	Manager	Manager: A manager can add a new customer.
	Manager	Manager: A manager can edit details like address, email, telephone of a customer.
New Account	Manager	Currently system provides 2 types of accounts • Saving • Current A customer can have multiple saving accounts (one in his name, other in a joint name etc). He can have multiple current accounts for different companies he owns. Or he can have a multiple current and saving accounts. Manager: A manager can add a new account for an existing customer.
Edit Account	Manager	Manager: A manager can add a edit account details for an existing account
Delete Account	Manager	Manager: A manager can add a delete an account for a customer.

Delete Customer	Manager	A customer can be deleted only if he/she has no active current or saving accounts Manager: A manager can delete a customer.
Deposit	Manager	Manager: A manager can deposit money into any account. Usually done when cash is deposited at a bank branch.
Withdrawal	Manager	Manager: A manager can withdraw money from any account. Usually done when cash is withdrawn at a bank branch.

1.1.2 Out of Scope

These feature are not be tested because they are not included in the software requirement specs

- User Interfaces
- Hardware Interfaces
- Software Interfaces
- Database logical
- Communications Interfaces
- Website Security and Performance

1.2 Quality Objective

The test objectives are to **verify** the Functionality of website Guru99 Bank, the project should focus on testing the **banking operation** such as Account Management, Withdrawal, and Balance...etc.
to **guarantee** all these operation can work **normally** in real business environment.

1.3 Roles and Responsibilities

The project should use **outsourced** members as the tester to save the project cost.

No.	Member	Tasks
1.	Test Manager	Manage the whole project Define project directions Acquire appropriate resources
2.	Test	Identifying and describing appropriate test techniques/tools/automation architecture Verify and assess the Test Approach Execute the tests, Log results, Report the defects. Outsourced members
3.	Developer in Test	Implement the test cases, test program, test suite etc.
4.	Test Administrator	Builds up and ensures test environment and assets are managed and maintained Support Tester to use the test environment for test execution
5.	SQA members	Take in charge of quality assurance Check to confirm whether the testing process is meeting specified requirements

2 Test Methodology

2.1 Overview

2.2 Test Levels

In the project Guru99 Bank, there're 3 types of testing should be conducted.

- **Integration** Testing (Individual software modules are combined and tested as a group)

- **System Testing:** Conducted on a **complete, integrated** system to evaluate the system's compliance with its specified requirements
- [/api-testing.html](#): Test all the APIs create for the software under tested

2.3 Bug Triage

2.4 Suspension Criteria and Resumption Requirements

If the team members report that there are **40%** of test cases **failed**, suspend testing until the development team fixes all the failed cases.

2.5 Test Completeness

- Specifies the criteria that denote a **successful** completion of a test phase
- **Run** rate is mandatory to be **100%** unless a clear reason is given.
- **Pass** rate is **80%**, achieving the pass rate is **mandatory**

2.6 Project task and estimation and schedule

Task	Members	Estimate effort
Create the test specification	Test Designer	170 man-hour
Perform Test Execution	Tester, Test Administrator	80 man-hour
Test Report	Tester	10 man-hour
Test Delivery		20 man-hour
Total		280 man-hour

Schedule to complete these tasks

3 Test Deliverables

Test deliverables are provided as below

Before testing phase

- Test plans document.
- Test cases documents
- Test Design specifications.

During the testing

- Test Tool Simulators.
- [Test Data](#)
- Test Trace-ability Matrix – Error logs and execution logs.

After the testing cycles is over

- Test Results/reports
- [Defect Report](#)
- Installation/ Test procedures guidelines
- Release notes

4 Resource & Environment Needs

4.1 Testing Tools

No. Resources Descriptions	
1. Server	Need a Database server which install MySQL server Web server which install Apache Server
2. Test tool	Develop a Test tool which can auto generate the test result to the predefined form and automated test execution
3. Network	Setup a LAN Gigabit and 1 internet line with the speed at least 5 Mb/s
4. Computer	At least 4 computer run Windows 7, Ram 2GB, CPU 3.4GHZ

4.2 Test Environment

It mentions the minimum hardware and software requirements that will be used to test the Application.

Following software's are required in addition to client-specific software.

- Windows 11 and above
- Office 2021 and above
- MS Exchange, etc.

What Is a Test Case in Software?

A test case is exactly what it sounds like: a test scenario measuring functionality across a set of actions or conditions to verify the expected result. They apply to any software application, can use manual testing or an [automated test](#), and can make use of test case management tools.

A key thing to remember when it comes to writing test cases is that they are intended to test a basic variable or task such as whether or not a discount code applies to the right product on an e-commerce web page. This allows a software tester more flexibility in how to test code and features.

Test Script vs. Test Case

The difference between test cases vs. test scripts should also be clarified. A test script is a short program meant to test certain functionality. A test case is a document with steps to be completed as planned out ahead of time.

Consider test cases as a meticulously planned trip and test scripts to be more like a quick trip to the grocery store.

Test Script vs. Test Case

The difference between test cases vs. test scripts should also be clarified. A test script is a short program meant to test certain functionality. A test case is a document with steps to be completed as planned out ahead of time.

Consider test cases as a meticulously planned trip and test scripts to be more like a quick trip to the grocery store.

Different Types of Test Cases

Test cases can measure many different aspects of code. The steps involved may also be intended to induce a Fail result as opposed to a positive expected result such as when a user inputs the wrong password on a login screen.

Some common test case examples would be the following:

Test cases can be applied to any number of features found in any given software. Some of the most popular include:

- [API testing example](#) – [See it in action.](#)
- [UI testing example](#) – [See it in action.](#)

- [Unit testing example](#) – [See it in action.](#)
- [Load & performance testing example](#) – [See it in action.](#)
- [Security testing](#)
- [SQL queries](#)
- [Low-code application testing](#)

A Popular Test Case Example

Test cases come in handy in a variety of software scenarios. Everything from banking to personal software requires a test case application. For example, if the goal is to have encrypted, sensitive data, the software needs to have features that work as intended.

But functional testing is just one aspect of writing a test case. Software testing should robustly challenge every aspect of the code from performance to compatibility to security. That's why personal encryption software needs to be tested so thoroughly — especially when it comes to [things like Web APIs](#).

How to Write Software Test Cases

Writing test cases varies depending on what the test case is measuring or testing. This is also a situation where sharing test assets across dev and test teams can accelerate software testing. But it all starts with knowing how to write a test case effectively and efficiently.

Test cases have a few integral parts that should always be present in fields. However, every test case can be broken down into 8 basic steps.

Step 1: Test Case ID

Test cases should all bear unique IDs to represent them. In most cases, following a convention for this naming ID helps with organization, clarity, and understanding.

Step 2: Test Description

This description should detail what unit, feature, or function is being tested or what is being verified.

Step 3: Assumptions and Pre-Conditions

This entails any conditions to be met before test case execution. One example would be requiring a valid Outlook account for a login.

Step 4: Test Data

This relates to the variables and their values in the test case. In the example of an email login, it would be the username and password for the account.

Step 5: Steps to be Executed

These should be easily repeatable steps as executed from the end user's perspective. For instance, a test case for logging into an email server might include these steps:

1. Open email server web page.
2. Enter username.
3. Enter password.
4. Click "Enter" or "Login" button.

Step 6: Expected Result

This indicates the result expected after the test case step execution. Upon entering the right login information, the expected result would be a successful login.

Step 7: Actual Result and Post-Conditions

As compared to the expected result, we can determine the status of the test case. In the case of the email login, the user would either be successfully logged in or not. The post-condition is what happens as a result of the step execution such as being redirected to the email inbox.

Step 8: Pass/Fail

Determining the pass/fail status depends on how the expected result and the actual result compare to each other.

Same result = Pass Different results = Fail

Accelerate Software Testing by Sharing Test Assets Across Dev & Test Teams

[Read the Blog](#)

Standard Unit Test Case Format

Each part of a [well-written unit test](#) will define several core aspects including:

1. Functions performed by the test
2. Data used in the test
3. Expected result from the test execution
4. Ensuring the test was executed in isolation from other parts of the codebase

It's important to know that the standard format of well-written tests is composed of the following parts:

- Meaningful test method name
- Controlled data or mocks to be used for testing
- Method or unit under test (the part of code we are testing)
- Applying an assertion
- Executing the unit test in isolation

Is There a Test Case Template?

As mentioned, there is a standard test case format. However, the test case template would likely vary from company to company and even from team to team. Instead, a test case template is the document with a list of test scenarios and subsequent test cases.

Quality Test Case Example

Though test cases will vary based on the type of testing and overall field of testing, building a quality test case comes down to those few reliable items above. Remember: the name of the test method must include the method or unit under test and what is the expected outcome.

It should also be noted that each unit should be tested in isolation. In this case, “isolation” means keeping tests focused as much as possible in order to execute only the piece of the application we are testing for.

This example comes from a banking-related test case:

With this method name, we know that this is a unit test that is:

- Testing the method ‘isOverDrawn()’.
- The balanced used for the controlled data was 500.
- The expected result is true.

A meaningful method name allows anyone reviewing the results to understand what the unit test was testing for. Moreover, it signals the data to be tested, the expected result, and what was tested.

If the test fails, knowing the expected result is critical in allowing for easier troubleshooting and [ensuring no regressions are introduced](#).

Test Case Data

The data used needs to be enough to execute the test. For unit testing, we want to make it as simple as possible to test the most basic unit of our application. The data could be as simple as making a string or object variable for which you can control the data. Or a mock framework can be used for the test if a dependency is not available or you need that dependency to be in a specific state.

Having just enough to test that one part is sufficient. You DO NOT need to configure every piece of the application for the test to run.

All of this affects how the unit test will behave since this is the data being used for unit test execution. As such, this part of unit testing is the most time consuming as it requires some understanding of the code you are testing to know what data to use for testing.

Keep it simple by using just the parts needed for the code being tested. Mocks are very useful in this phase as they allow you to control how methods from those objects will behave when interacting with your test.

For example, given the following data:

We avoided the “real customer class” by using a mock for the “customer class” for testing isolation. We do not want to introduce nor configure another object for this test as it adds another layer of maintainability for that object, and it is not affecting the result of the method under test.

The next variable to be created is the “initial balance”—something known due to knowledge of the code. The next line shows the Account object being created along with the mock and the Initial Balance to prepare the method we are testing for with the data we just used.

So in this example, the account object is configured with the mock customer since we do not care about the customer object’s data and we passed an initial balance that we can control for our test.

The next line defines the input as the method under test requires a number to be used. We defined the “balance” to be used in the method we are testing for. Then the method is executed with the result of the method being stored in our variable for us to use later.

Applying an Assertion

Once the test can complete successfully (as in it runs from start to finish with no exceptions or errors), then it is time to apply an assertion to the unit test. Without the assertion, the unit test is meaningless since there is nothing you are enforcing to ensure it is working as intended.

Collecting coverage of what lines were executed does tell what was executed but it does not provide enough detail to determine the following:

- If the code is behaving as expected.
- If the code meets quality targets.
- If the data returned is the expected data.

An assertion can be as basic as:

As long as the unit test contains one assertion that is checking the method under test result, this is a meaningful unit test.

By applying the standard format of unit test, a team can easily maintain, read, and/or update tests with more ease to readily see where more testing can be applied to the rest of the application.

What Are the Best Practices for Writing Quality Test Cases?

How to write effective tests and test cases can be streamlined over time. Some [best practices](#) include using strong titles, strong descriptions, and keeping the language concise and clear.

But you'll also want to include preconditions, assumptions, and the expected results, too. All of this information is relevant for the software tester — especially when determining whether the test case should be a “pass” or a “fail” instead.

A cheat sheet for creating test cases that work well is as follows:

- Keep things simple and transparent.
- Make test cases reusable.
- Keep test case IDs unique.
- Peer review is important.
- Test cases should have the end user or defined requirements in mind.
- Specify expected results and assumptions.

Simple, unique, specific, open to feedback, and focused on reusability: that's the way of a great test case. For a more visual look at how to write a quality test case, check out Parasoft's webinar on the subject.

Test Suite vs. Test Plan

The other aspect of a test case involves test suites and test plans. These differ in key ways and both are vital to accurate test case development.

Improve Unit Testing for Java With Automation

[Get the Ebook](#)

[What Is a Test Suite?](#)

A test suite comes into play for test cases as it relates to source code, the collection of dependencies, or the suite of tests to be performed on code. Test

suites allow you to categorize test cases in ways that align with any analysis or planning needs.

This means that core software features might have their own test suite while another test suite is for a specific testing type such as smoke or security. Think of test suites as a bookshelf to organize your test cases on.

What Is a Test Plan?

In contrast, a test plan is more like the umbrella that stands over all of the test suites. If test cases are books and test suites are bookshelves, then test plans are the room that contains the bookshelf.

Generally, test plans are set up in terms of manual tests, automated tests, and a general format of how to go about testing. They'll test the software from the foundation up utilizing test suites and test cases before implementing changes or adding new features.

Prepare any eight test cases for admission form of college admission.

Test Case Id	Test case Objectives	Input Data	Expected Result	Actual Result	Status
TC1	Name field	Any name (abcd xyz)	It should accept the name	The name is accepted	Pass
TC2	Phone Field	Any number Having less than 10 digits(1234)	It should not Accept. Should give error message "Please enter valid phone number"	Error message "Please enter valid phone number"	Pass
TC3	Phone Field	Any Alphabets (abcde)	It should give error message as "Only Numbers"	Error message as "Only Numbers"	Pass
TC4	SSC Percentage Field	65	It should accept	It accepted	Pass
TC5	SSC Percentage Field	30	It should not accepted should give error message	Gives error message	Pass
TC6	Address field	Any characters (A-51, Market road, Mumbai)	It should accept	It accepted	Pass

